

Nome e cognome: _____ Classe: _____ Data: _____

Esercizio: Roster Atleti

TypeScript · moduli con import/export · DOM · localStorage

1. Scenario

Devi gestire il roster di una gara a squadre in stile Physical. Ogni atleta ha un nome, una disciplina e una foto. L'applicazione permette di aggiungere atleti, segnare chi è in squadra e chi è in riserva, filtrare l'elenco ed eliminare un atleta: quando l'atleta viene eliminato, sparisce anche la sua foto.

È lo stesso schema della todo list vista in classe: cambiano i dati (un atleta al posto di un'attività) e la scheda da disegnare (una card con immagine).

2. Obiettivi

- Suddividere un programma in più file, ognuno con una sola responsabilità.
- Usare `export` e `import` (anche `import type`) senza creare dipendenze circolari.
- Mantenere lo stato in un solo modulo e modificarlo solo tramite funzioni.
- Creare elementi HTML da codice, comprese le immagini.
- Salvare e ricaricare i dati con `localStorage`.

3. Avvio del progetto

Estrai lo zip del progetto di partenza, apri la cartella nel terminale ed esegui:

```
npm install
npm run dev
```

Apri l'indirizzo mostrato nel terminale (di solito `http://localhost:5173`). Per controllare gli errori di tipo puoi usare `npx tsc`.

4. Struttura dei file

Tutti i file sono in `src/`. Nei file da completare trovi dei commenti `TODO` che indicano cosa scrivere.

File	Stato	Cosa deve contenere
<code>types.ts</code>	Già pronto	Interfaccia Atleta e tipo Filtro.
<code>storage.ts</code>	Da completare	<code>salva(atleti)</code> e <code>carica()</code> con <code>localStorage</code> .
<code>state.ts</code>	Da completare	Lo stato (atleti e filtro corrente) e le funzioni per modificarlo: <code>aggiungiAtleta</code> , <code>eliminaAtleta</code> , <code>impostaInSquadra</code> , <code>impostaFiltro</code> , <code>getAtletiVisibili</code> .
<code>render.ts</code>	Da completare	<code>renderAtleti()</code> , che disegna una card per ogni atleta visibile.
<code>main.ts</code>	Da completare	Solo eventi: collega il form e i filtri agli altri moduli e fa il primo render.

5. Dati

```
interface Atleta {
  id: number;
```

```
nome: string;
disciplina: string;
foto: string;          // URL dell'immagine
inSquadra: boolean;
}

type Filtro = "tutti" | "inSquadra" | "riserve";
```

6. Requisiti

1. Il form ha tre campi (nome, disciplina, URL della foto) e il pulsante Aggiungi. Se nome o URL sono vuoti, non si aggiunge nulla.
2. Ogni atleta è una card con foto (), nome, disciplina, una checkbox “In squadra” e un pulsante “Elimina”.
3. Se l'atleta è in squadra, la card ha la classe CSS in-squadra (già definita nello stile).
4. Tre pulsanti filtro: Tutti, In squadra, Riserve.
5. Eliminando un atleta, sparisce dalla pagina la sua card, foto compresa, e anche dai dati salvati.
6. Ricaricando la pagina i dati restano.
7. main.ts non contiene logica sui dati: solo eventi e chiamate agli altri moduli.
8. Nessun import circolare tra i file.

7. Bonus (in ordine di difficoltà)

- Foto di riserva: se l'immagine non si carica, mostra un'immagine alternativa (evento `error` su `<img>`).
- Prima di eliminare, chiedi conferma con `confirm()`.
- Contatore “N atleti in squadra su M” nel paragrafo `#contatore`, sempre aggiornato.
- Caricamento della foto da file con `FileReader` e `readAsDataURL`, salvandola nei dati dell'atleta. Attenzione al limite di circa 5 MB del `localStorage`.

8. Suggerimenti

- Per le foto puoi usare URL segnaposto (ad esempio `https://picsum.photos/200`) oppure immagini messe nella cartella `public/`. Non usare foto di persone reali.
- Se una variabile esportata con `let` viene riassegnata in un altro file, TypeScript dà errore: per questo lo stato si modifica con funzioni.
- Lavora a piccoli passi: prima aggiungi e mostra gli atleti, poi checkbox ed eliminazione, poi filtri e `localStorage`.
- Dopo ogni modifica ai dati ricordati di richiamare `renderAtleti()`.

9. Valutazione

Criterio	Punti	Ottenuti
Ogni file ha una sola responsabilità, import corretti e senza cicli	3	
Stato modificato solo tramite le funzioni di state.ts	2	
Aggiunta, filtri ed eliminazione (con foto) funzionano	3	
Persistenza con localStorage	1	
Nomi chiari e codice leggibile	1	
Totale	10	

I bonus possono aggiungere fino a 1 punto ciascuno, a discrezione dell'insegnante.